

Analiza uszkodzeń radiacyjnych w białkach i ligandach krystalograficznych

ZBOiB — RD, Bioinfo 2

Cele zajęć

Student:

- pozna źródła uszkodzeń radiacyjnych (ang. radiation damage, RD) i ich markery w mapach elektronowej gęstości,
- nauczy się pobierać mapy 2Fo–Fc i Fo–Fc z PDBe i wczytywać je do PyMOL,
- przeprowadzi analizę struktur białek i ligandów narażonych na wysoką dawkę promieniowania,
- przeanalizuje dwa rzeczywiste przypadki: Iodo-willardiine (PDB 3T96) oraz Baclofen (PDB 4MS4),
- stworzy skrypt w PyMOL, który zautomatyzuje proces tworzenia scen z potencjalnymi markerami RD
- przygotuje mini-raport wykazujący trzy zidentyfikowane uszkodzenia radiacyjne.

1 Wprowadzenie teoretyczne

Uszkodzenia radiacyjne w krystalografii makromolekularnej wynikają z interakcji wysokoenergetycznych fotonów z kryształem białka. Procesy wstępne obejmują:

- fotojonizację,
- emisję fotoelektronów,
- generację elektronów wtórnych,
- radiolizę wody i tworzenie rodników.

Najczęstsze uszkodzenia białek:

- dekarboksylacja **Asp/Glu**,
- pękanie mostków **Cys–Cys**,
- utlenianie **Met** (zanik gęstości na SD),
- zaburzenia pierścieni aromatycznych **Tyr/Trp**.

Rzadziej omawiane, ale kluczowe dla drugiej części zajęć:

- uszkodzenia *ligandów*: odrywanie halogenów, redukcje grup nitrowych, przesunięcia pierścieni aromatycznych, utrata końcowych atomów.

Manifestacja w mapach Fo–Fc:

- **ujemna gęstość** — utrata atomu lub jego przesunięcie,
- **dodatnia gęstość** — alternatywne położenie lub nowy konformer,
- **brak gęstości** — całkowity zanik grupy.

2 Przygotowanie środowiska i pobranie danych

2.1 Pobieranie struktur

Dwie analizowane struktury:

- **3T96** — AMPA receptor + Iodo-willardiine,
- **4MS4** — GABA_B receptor + Baclofen.

Proszę pobrać pliki ze strukturą badanych białek. W tym celu:

- (a) Wejdź na stronę RCSB PDB: <https://www.rcsb.org>.
- (b) W polu wyszukiwania wpisz odpowiednio identyfikatory 3T96 oraz 4MS4.
- (c) Przejdź do strony danej struktury i zapisz lokalnie plik w formacie **mmCIF** (zalecany) lub PDB.

Alternatywnie można wykorzystać wbudowaną w PyMOL funkcję `fetch`, która pobiera struktury bezpośrednio z PDB (zob. dokumentację komendy `fetch` w PyMOL, np. <https://pymolwiki.org/index.php/Fetch>). Przykład użycia:

```
fetch 3T96, type=cif, async=0
fetch 4MS4, type=cif, async=0
```

Zachęcam, aby świadomie zdecydować, z której metody skorzystać (przeglądarka vs `fetch`) i rozumieć, skąd pochodzą dane wejściowe.

Dla większej czytelności warto otworzyć każdą strukturę w osobnym oknie Pymola.

2.2 Pobieranie map gęstości elektronowych

W przypadku wielu struktur krystalograficznych RCSB nie udostępnia gotowych plików map elektronowej gęstości w formacie `.map`. Zamiast tego dostępne są pliki *współczynników mapy* (tzw. *map coefficients*).

Aby móc pracować z mapami w PyMOL, należy najpierw wygenerować właściwą mapę (CCP4/MRC) z użyciem programu **gemmi**. Gemmi jest lekką, szybką biblioteką i narzędziem CLI do pracy z danymi krystalograficznymi.

W naszym przypadku będziemy pobierać wcześniej przekonwertowane do właściwego formatu (ccp4) mapy:

- Pliki z mapami gęstości proszę pobrać z platformy Pegaz i umieścić w katalogu, w którym będzie uruchamiany Pymol

2.3 Wczytanie struktur w PyMOL

Proszę uruchomić PyMOL i załadować uprzednio pobrane pliki. Można to zrobić:

- wykorzystując wcześniej poznaną funkcję **fetch**
- przeciągając pliki `.cif` do okna PyMOL,
- lub używając konsoli PyMOL (co jest zalecane ze względu na powtarzalność).

Przykładowe komendy (przy założeniu, że pliki znajdują się w bieżącym katalogu roboczym):

```
load 3T96.cif
load 4MS4.cif
bg_color white
hide everything
show cartoon
color grey70
```

Zachęcam, aby:

- nadać obiektom nazwy, które są dla Ciebie czytelne (np. `ampa`, `gabab`),
- samodzielnie wypróbować inne reprezentacje (`surface`, `sticks`) dla lepszego zrozumienia budowy kompleksu.

2.4 Wczytanie i wyświetlenie ligandów

Kolejnym krokiem jest zlokalizowanie i wizualizacja ligandów w obu strukturach. Proszę:

- (a) Sprawdzić w opisie struktury (nagłówek PDB/mmCIF, strona RCSB), pod jakim **resn** zapisano dany ligand.
- (b) Przy pomocy selekcji w PyMOL wyodrębnić ligand i nadać mu odrębną reprezentację.

Dla Iodo-willardiine (3T96), ligand jest zapisany jako `IWD`. Przykład:

```
select will, resn IWD
show sticks, will
util.cnc
```

Analogicznie dla Baclofenu (4MS4) — ligand ma inne oznaczenie:

```
select bac, resn NAZWA_RESZTY_LIGANDA
show sticks, bac
```

Proszę upewnić się, że wybór ligandów jest poprawny (porównując np. opis w RCSB z tym, co widzimy w PyMOL).

2.5 Wczytanie map

Po załadowaniu struktur należy dodać mapy elektronowej gęstości. Proszę:

- sprawdzić, w którym katalogu znajdują się pliki `.map`,
- upewnić się, że PyMOL „widzi” te pliki (komenda `pwd`, ewentualnie `cd` do katalogu z mapami),
- załadować mapy przy pomocy komendy `load`.

Przykład:

```
load 3t96_2fo.map
load 3t96_fo.map
load 4ms4_2fo.map
load 4ms4_fo.map

# przykładowa wizualizacja siatek dla 3T96
isomesh t96_2fo, 3t96_2fo, 1.0, 3t96, carve=2.0
isomesh t96_fo_pos, 3t96_fo, 3.0, 3t96, carve=2.0
isomesh t96_fo_neg, 3t96_fo, -3.0, 3t96, carve=2.0
set transparency, 0.6, t96_2fo
# standardowe kolory dla map
color forest, t96_2fo
color red, t96_fo_neg
color blue, t96_fo_pos
```

Zachęcam, aby:

- poeksperymentować z wartościami `carve` i wyjaśnić za co odpowiada ten parametr (ostatni argument `isomesh`),
- sprawdzić różnicę między mapą 2Fo–Fc (gęstość „łączna”) a Fo–Fc (mapa różnicowa, w której widzimy m.in. skutki uszkodzeń).

Analogicznie dla Baclofenu (4MS4), proszę nazywać mapy:
`ms4_2fo`, `ms4_fo_pos`, `ms4_fo_neg`

2.6 Kontrola jakości modeli

Na tym etapie proszę wykonać prostą kontrolę jakości dla obu modeli. W szczególności:

- obejrzeć rozkład B-faktorów (np. `spectrum b`, `blue_white_red`),
- zwrócić uwagę które miejsca w białku charakteryzują się wysokimi wartościami czynników temperaturowych,
- porównać to, co widać w modelu, z tym, co sugeruje mapa 2Fo-Fc (czy atomy rzeczywiście „siedzą” w gęstości).

W notatkach do ćwiczenia proszę zapisać co najmniej jedno miejsce, które już na tym etapie wygląda podejrzanie (może wskazywać na uszkodzenie radiacyjne lub słabą jakość modelu).

3 Analiza uszkodzeń radiacyjnych białka

3.1 Identyfikacja podejrzanych reszt

Uszkodzenia radiacyjne częściej dotyczą konkretnych typów reszt. Proszę przygotować selekcje grup reszt szczególnie narażonych:

```
select acidic,      resn ASP+GLU
select cysteines, resn CYS
select meth,        resn MET
select arom,         resn TYR+TRP+PHE
```

Następnie:

- kolejno oglądać reszty z każdej selekcji w kontekście map Fo-Fc,
- zwłaszcza zwrócić uwagę na końce łańcuchów bocznych (COO⁻, SD Met, pierścienie aromatyczne),
- notować, gdzie pojawia się silna **ujemna** gęstość w miejscu, gdzie „powinien” być atom.

3.2 Przegląd map Fo-Fc

Podczas przeglądu proszę zwrócić uwagę na:

- ujemne piki na końcach łańcuchów karboksylowych (dekarboksylacja Asp/Glu),
- zanik gęstości SD Met (utlenianie siarki),
- zrywanie mostków disiarczkowych (Cys-Cys),
- „dziury” w pierścieniach aromatycznych Tyr/Trp (częściowa utrata pierścienia lub błędnie dopasowany model).

Student zapisuje **co najmniej trzy** przykłady podejrzanych miejsc (na razie tylko w białku), z krótkim opisem:

- która reszta (łańcuch, numer, resn),
- jaki rodzaj anomalii w mapie,
- jaka może być interpretacja (rodzaj uszkodzenia).

4 Uszkodzenia ligandów

4.1 Iodo-willardiine (3T96)

Ligandy również mogą ulegać uszkodzeniom radiacyjnym. W przypadku Iodo-willardiine szczególnie interesujący jest atom jodu oraz fragmenty w jego sąsiedztwie. Proszę:

- przybliżyć obszar liganda,
- nałożyć mapy Fo–Fc,
- poszukać ujemnych pików w pobliżu atomu I oraz końców łańcucha.

Typowe artefakty, na które warto zwrócić uwagę:

- **ujemna Fo–Fc w pobliżu atomu I** — możliwe częściowe odrywanie halogenu,
- przesunięcia atomów w pierścieniu (dodatnia/ujemna gęstość wokół pierścienia),
- fragmentacja końca łańcucha (brak gęstości na częściach liganda).

Przykładowe ustawienia w PyMOL:

```
zoom will
hide mesh, t96_*
isomesh will_2fo, 3t96_2fo, 1.0, will, carve=4.0
isomesh will_fo_pos, 3t96_fo, 3.0, will, carve=4.0
isomesh will_fo_neg, 3t96_fo, -3.0, will, carve=4.0
```

Proszę zwiększać carve dla map różnicowych, jeśli nie obejmuje całego uszkodzenia. Proszę zanotować przynajmniej jedno miejsce w ligandzie, gdzie mapa sugeruje uszkodzenie.

4.2 Baclofen (4MS4)

Dla Baclofenu proszę wykonać analogiczną analizę:

- przybliżyć ligand (`zoom bac, 8`),
- obejrzeć mapy 2Fo–Fc oraz Fo–Fc,
- zwrócić uwagę na końce bocznych łańcuchów, pierścień aromatyczny oraz atomy O/N.

Możliwe uszkodzenia:

- utrata końców bocznego łańcucha (ujemna gęstość, brak gęstości 2Fo–Fc),

- rotacyjne alternatywy aromatu (dodatnia gęstość w alternatywnej pozycji pierścienia),
- przesunięcia atomów O/N (ujemna gęstość w modelu, dodatnia w innym miejscu).

Proszę wskazać co najmniej jedno miejsce, w którym ligand sprawia wrażenie „niedopasowanego” do gęstości, i spróbować zinterpretować to jako potencjalny skutek uszkodzenia radiacyjnego lub błędu modelowania.

5 Skrypt do automatycznego tworzenia scen

W tej części zautomatyzujemy tworzenie scen w PyMOL dla wszystkich miejsc potencjalnych uszkodzeń radiacyjnych, które zidentyfikowałeś wcześniej.

5.1 Założenia skryptu

Zamiast ręcznie:

- przybliżać każde miejsce,
- nakładać mapy,
- zapisywać scenę,

napiszemy skrypt w Pythonie (uruchamiany w PyMOL), który zrobi to automatycznie dla wszystkich reszt w jednej selekcji (np. `damage_res`). Celem jest utrwalenie prostego sposobu automatyzacji powtarzalnych zadań w analizie struktur.

5.2 Przygotowanie selekcji w PyMOL

Najpierw, na podstawie poprzednich części, utwórz selekcję zawierającą wszystkie miejsca, które uznałeś za potencjalne uszkodzenia radiacyjne (zarówno w białku, jak i w ligandach). Przykład:

```
# PRZYKŁAD - dostosuj do swoich wyników
select damage_res, (3t96 within 4 of will)
# dodatkowo możesz ręcznie dodawać konkretne reszty, które przeanalizowałeś:
select damage_res, damage_res or (3t96 and resi 150 and chain A)
select damage_res, damage_res or (4ms4 and resi 210 and chain B)
select damage_res, damage_res or cysteines
```

Uwaga: ważne jest, aby selekcja `damage_res` zawierała atomy tych reszt, które chcesz obejrzeć w scenach. W praktyce warto zacząć od mniejszej liczby miejsc i stopniowo ją rozszerzać.

5.3 Szkielet skryptu Python (do uzupełnienia)

Utwórz plik `damage_scenes.py` z poniższym szkieletem:

```
from pymol import cmd

def make_damage_scenes(sel_name, obj_name, map_2fofc, map_fofc,
                      around=4.0, level_2fofc=1.0, level_fofc=3.0):
    """
    Tworzy sceny w PyMOL dla wszystkich unikalnych reszt w selekcji sel_name.
    Dla każdej reszty:
    - przybliża obszar
    - pokazuje cartoon białka i sticks dla reszty
    - nakłada mapy 2Fo-Fc i Fo-Fc
    - zapisuje scenę o nazwie damage_XX_RESIDUE
    """
    # Pobierz model wybranych atomów
    model = cmd.get_model(sel_name)
    seen = set()
    idx = 1

    for at in model.atom:
        key = (at.segi, at.chain, at.resi, at.resn)
        if key in seen:
            continue
        seen.add(key)

        # TODO (1): zbuduj selekcję dla całej reszty (podpowiedź: użyj obj_name,
        # at.chain i at.resi)
        # np. sele = f"/{obj_name}/{at.chain}/{at.resi}/"
        sele = f"/{obj_name}/{at.chain}/{at.resi}/"

        # Wyczyść scenę i ustaw reprezentacje
        cmd.hide("everything")
        cmd.show("cartoon", obj_name)
        cmd.show("sticks", sele)

        # Przybliżenie
        cmd.zoom(sele, around)

        # TODO (2): nałóż mapy 2Fo-Fc i Fo-Fc jako siatki
        # (podpowiedź: isomesh, level_2fofc, level_fofc)
        cmd.isomesh("m2", map_2fofc, level_2fofc, sele)
        cmd.isomesh("mf", map_fofc, level_fofc, sele)

        # Nazwa sceny: damage_01_RES123A
        scene_name = f"damage_{idx:02d}_{at.resn}{at.resi}{at.chain}"
        cmd.scene(scene_name, "store")
        print(f"Zapisano scenę: {scene_name}")
        idx += 1
```



```
# Rejestracja komendy PyMOL:
cmd.extend("make_damage_scenes", make_damage_scenes)
```

Zadanie:

- uzupełnij oznaczone miejsca TODO (jeśli jeszcze tego nie zrobiłeś),
- uruchom skrypt w PyMOL komendą `run damage_scenes.py`,
- przeanalizuj, czy wygenerowane sceny odpowiadają miejscom, które wcześniej zidentyfikowałeś.

5.4 Uruchomienie skryptu w PyMOL

Przykład wywołania dla struktury **3T96**:

```
# zakładamy, że:
# - białko nazywa się "t96"
# - mapy nazywają się "t96_2fofc" i "t96_fofc"
# - selekcja z uszkodzonymi miejscami to "damage_res"
```

```
make_damage_scenes sel_name="damage_res",
                   obj_name="t96",
                   map_2fofc="t96_2fofc",
                   map_fofc="t96_fofc"
```

Analogicznie można wywołać funkcję dla **4MS4**, podając odpowiednie nazwy obiektu i map.

5.5 Analiza wygenerowanych scen

Po wykonaniu skryptu:

- użyj panelu **Scenes** w PyMOL, aby przechodzić między widokami,
- sprawdź, czy wszystkie sceny rzeczywiście odpowiadają miejscom, które wcześniej uznałeś za potencjalne uszkodzenia,
- zastanów się, czy poziomy konturu map (parametry `level_2fofc`, `level_fofc`) są optymalne dla Twoich danych, i w razie potrzeby zmodyfikuj je.

5.6 Raport

Każdy student przygotowuje mini-raport (max 2 strony, PDF), który zawiera:

- trzy miejsca uszkodzeń w białku (dowolna z analizowanych struktur),
- jedno miejsce uszkodzenia w każdym ligandzie (Iodo-willardiine, Baclofen),
- zrzuty ekranu z map 2Fo–Fc i Fo–Fc ilustrujące wskazane uszkodzenia,
- krótką interpretację (*rodzaj uszkodzenia + dowód w mapie*),

- zrzut ekranu z scenami wygenerowanymi przez skrypt,
- szczegółowe wyjaśnienie wszystkich komend (w j.pol.) wykorzystanych w części 2 i 3.

System oceniania (10 punktów)

- **1 pkt** — poprawne wczytanie i prezentacja map,
- **3 pkt** — poprawnie napisany i uruchomiony skrypt (lub dobrze zdiagnozowane problemy),
- **2 pkt** — identyfikacja uszkodzeń w białku,
- **2 pkt** — identyfikacja uszkodzeń w ligandach,
- **2 pkt** — jakość raportu (jasność, zrzuty, interpretacja).